

SIMATS ENGINEERING**ASSESSMENT TOOL 1: Data Interpretation****COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A****COURSE OUTCOME COVERED****CO4: Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL 5) Assessment Tool Weightage: 40%****QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A****Data Interpretation****Code of Conduct:**

I D.HUSSAINIAH(192425300) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.HUSSAINIAH**Rubrics for Evaluation**

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	

Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

Read the given data carefully and compare the techniques or results based on multiple factors such as accuracy, robustness, and computational constraints. Evaluate and justify your decision with appropriate reasoning considering the application context.

1. Real-Time System Design Trade-off

A surveillance system must operate under constraints:

- Viola-Jones: fast but less accurate (78%)
- YOLO: moderate speed, good accuracy (92%)
- Deep Learning: high accuracy (95%) but slow

Compare the techniques and evaluate which should be selected under strict latency constraints while maintaining acceptable accuracy. Justify your decision considering deployment conditions.

Answer:

Method	Speed	Accuracy	Where it fits
Viola-Jones	Very fast, runs fine on CPU	78%	Cheap embedded devices where speed matters more than precision
YOLO	Fast enough for live video	92%	Real-time surveillance needing a balance of both
Deep Learning	Slow, needs a GPU to keep up	95%	Offline review or high-end hardware setups

For a surveillance feed that has to run live, Viola-Jones is out of the picture even though it's the fastest option — a 78% accuracy rate means roughly one in five detections could be wrong, which isn't something you want in a security system. Deep Learning sits at the other end: 95% accuracy is great, but on a live camera feed the processing lag alone could cause frames to be dropped or the system to fall behind real time.

That leaves YOLO as the practical middle ground. It's a single-pass detector, so it's built for exactly this kind of scenario — good enough accuracy (92%) without the heavy latency of a full deep network. Given the constraint here is 'strict latency', YOLO should be selected. If the deployment site does happen to have GPU/edge-AI hardware available, a lighter deep learning model could be considered later to push accuracy closer to 95%, but as a default choice under latency pressure, YOLO wins.

2. Robust Detection under Environmental Variations

Detection accuracy across environments:

Method Normal Occlusion Low Light

Viola-Jones 80% 60% 55%

YOLO 90% 75% 70%

Deep Learning 95% 85% 80%

Compare the robustness of each method and evaluate which technique provides the best overall performance across varying conditions. Justify trade-offs.

Detection accuracy across environments:

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Compare the robustness of each method and evaluate which technique provides the best overall performance across varying conditions. Justify trade-offs.

Answer:

Looking at how much each method's accuracy falls off as conditions get harder tells the real story here, not just the normal-condition number:

- Viola-Jones falls from 80% to 55% — a 25-point drop. It depends on handcrafted Haar features, which are basically edge/intensity patterns tuned for good lighting and unobstructed faces/objects, so it breaks down quickly once either of those is taken away.
- YOLO falls from 90% to 70% — a 20-point drop. Better than Viola-Jones because it learns its features from data instead of using fixed rules, but it's still clearly affected by occlusion and darkness.
- Deep Learning falls from 95% to 80% — only a 15-point drop, and it stays on top in every single condition.

So Deep Learning is the most robust of the three, both because it scores highest everywhere and because it degrades the least when things get difficult. That's expected — deeper networks build up layered features (edges, then parts, then whole objects) that don't rely on any one visual cue, so losing part of the object to occlusion or losing contrast in low light doesn't break the detection as easily.

The catch is cost. Robustness this good comes from a heavier model that needs more compute, which ties back to the latency problem in Question 1. i.e. paying the extra computational cost for Deep Learning.

3. Optical Flow Reliability vs Noise Sensitivity

Two optical flow methods show:

- Method A: stable vectors but misses small motions
- Method B: detects fine motion but noisy vectors

Compare both methods and evaluate which is more suitable for motion tracking in dynamic scenes. Justify based on application requirements.

Answer:

These two methods are basically opposite failure modes of the same problem. Method A is probably using a larger window or heavier smoothing when it estimates flow, which gives clean, stable vectors but washes out anything subtle — small or slow movements just get averaged away. Method B is the opposite: it's sensitive enough to pick up fine motion, but that same sensitivity means it also reacts to noise, lighting flicker, and compression artifacts, so its vector field is jittery.

For general motion tracking in a dynamic scene — say, tracking people or vehicles across a video — stability usually matters more than catching every tiny movement. Noisy vectors from Method B can actually break a tracker: an object's ID can jump around, trajectories look shaky, and any downstream prediction (like estimating where an object is heading) becomes unreliable. So for most tracking applications, Method A is the safer default.

That said, Method B shouldn't be thrown out — if the application specifically needs to catch fine or slow motion (vibration monitoring, gesture recognition, early motion detection), its sensitivity is exactly what's needed, and the noise problem can be handled separately with a smoothing step (median filtering, Kalman filtering, etc.). In practice, combining the two — Method B for detection sensitivity, cleaned up with the kind of temporal smoothing Method A effectively already does — gives the best of both. But if I had to pick one method as-is for dynamic scene tracking, I'd go with Method A.

4. Background Modeling Strategy (GMM)

A GMM-based system shows:

- High false positives in dynamic background
- Missed detections in static background

Compare GMM performance across scenarios and evaluate whether GMM alone is sufficient. Propose and justify improvements.

Answer:

GMM models each pixel's history as a mixture of Gaussians and flags a pixel as foreground when it no longer fits that learned distribution. The two problems described here actually come from the same root cause — the model isn't flexible enough to tell the difference between 'background that just looks like it's changing' and 'something new that has actually entered the scene.'

In a dynamic background — trees moving in the wind, rippling water, an escalator — the pixel intensities are genuinely fluctuating all the time, and GMM ends up treating a lot of that natural variation as foreground, which is where the false positives come from. On the flip side, in a static background, if an object stops moving or moves very slowly, GMM's own background-update mechanism gradually folds it into the background model — this is the classic 'sleeping object' problem — and it stops being detected at all.

So no, GMM on its own isn't really sufficient outside of a fairly controlled scene with a genuinely static background and clearly moving foreground objects. Once real-world conditions creep in (wind, water, people pausing), plain GMM starts failing in both directions at once.

A few things would help:

- Use more Gaussian components per pixel with a properly tuned adaptive learning rate, so recurring background motion (leaves, water) gets absorbed into the model instead of being flagged.
- Add morphological cleanup — erosion/dilation and small-blob removal — to strip out the scattered false-positive noise from dynamic regions.
- Use two update rates instead of one: a fast one for the general background and a slower one so that an object which pauses briefly isn't immediately swallowed into the model.
- Add a shadow-suppression step, since moving shadows around dynamic elements are another common

source of false positives.

- Pair GMM with a lightweight object detector (YOLO, for instance) as a verification layer — GMM proposes candidate regions cheaply, and the detector confirms or rejects them, catching both the false positives and the objects GMM alone would have missed.

Together, these turn GMM from a standalone (and fairly fragile) detector into the first stage of a more reliable pipeline.

5. Motion Estimation vs Computational Cost

Two methods:

- Method A: low error (5%), high computation
- Method B: moderate error (10%), low computation

Compare the methods and evaluate their suitability for real-time vs offline applications. Justify your decision.

Answer:

Aspect	Method A	Method B
Error	5% — quite accurate	10% — acceptable, not great
Computation	Heavy per frame	Light, runs quickly
Fits best in	Offline / batch processing	Real-time systems

This one comes down to a straightforward speed-vs-accuracy call. Method A gets you the lowest error, but the computational cost means it's unlikely to keep up with a live video feed unless there's serious hardware behind it — so it makes more sense for offline work: forensic analysis, research evaluation, anything where the video is processed after the fact and there's no deadline on each frame.

Method B gives up some accuracy (10% vs 5%) but is light enough to run in real time, which makes it the better fit for anything with a live constraint — robotics control, live tracking, driver-assistance systems — where a slightly higher error rate is a fair trade for not falling behind the video stream.

If a system genuinely needs both — real-time responsiveness and high accuracy — a reasonable middle path is to run Method B continuously for the live output, and periodically cross-check or refine using Method A on selected keyframes rather than every frame. But for a straight real-time vs offline decision as asked, Method B goes with real-time and Method A goes with offline.